About this PDF

Built only from: **NOC:Cloud Computing and Distributed Systems (NPTEL course book, weeks 1, 3)** — https://nptel.ac.in/courses/106104182

Licence: CC BY-SA 4.0. Accessed 2026-10-09.

Covers 8 of 11 syllabus sub-topics of this unit.

Partly covered in the source:

- Introduction: Definition-Relation to Computer System Components
- Primitives for Distributed Communication
- Synchronous versus Asynchronous Executions
- A Model of Distributed Executions
- Models of Communication Networks
- Global State of a Distributed System.

Not in the source (not covered in this PDF):

- Motivation
- Passing Systems versus Shared Memory Systems
- Design Issues and Challenges

Introduction: Definition-Relation to Computer System Components

Partly covered in the source.

Definition

Cloud computing is a model that provides scalable computing over the internet, evolving from earlier distributed systems.

Explanation

It differs from previous generations of computing systems by focusing on data-intensive and network-centric workloads, driven by applications with variable workloads and large data sets. The evolutionary changes that have occurred in distributed and cloud computing over the past 30 years are driven by applications, with changes in machine architectures, operating systems, platforms, network connectivity and application workloads. Leader election is a critical component in many cloud computing applications and systems, used for coordination.

Example

High-throughput computing systems such as computing clusters, service-oriented architecture, computational grids, peer-to-peer networks, internet cloud, and the future of the Internet of Things illustrate cloud computing. In a leader election scenario, if the current leader N80 fails, its successor N32 monitors it and becomes the new leader if N80 does not respond.

Algorithm / Steps

1. Each process monitors its next higher-ID process.

2. If the successor (leader) fails, the monitoring process becomes the new leader.
3. If the successor responds, wait for a timeout and check the next successor.
4. If that successor also fails, the monitoring process becomes the leader.

Key points

- Cloud computing provides scalable computing over the internet, evolving from distributed systems.
- It addresses data-intensive and network-centric workloads with variable workloads and large data sets.
- Leader election is essential for coordination in cloud systems.
- In leader election, a process monitors its successor and becomes leader if the successor fails.
- High-throughput computing systems illustrate cloud computing applications.

Message

Definition

In the leader election protocol, a message is a communication unit that carries information such as a process's identifier or a status flag. For example, an election message contains the sender's own id as its attribute, while a coordinator message announces the elected leader's id.

Explanation

When a process initiates an election, it sends an election message to processes with higher identifiers. Each receiving process compares the id in the message with its own. If the received id is larger, it forwards the message unchanged; if it is smaller and the process has not forwarded an election message before, it replaces the id with its own and forwards it. When the message returns to the process that originally sent it, that process changes the message type to elected and circulates it so that all processes learn the leader. If a process detects that the coordinator has failed via a failure detector, it checks whether its own id is the highest among known processes. If so, it elects itself as coordinator and sends a coordinator message to all processes with lower ids. If its id is not the highest, it initiates an election by sending an election message containing its id to all processes with higher ids. If a process receives no reply to its election message within a timeout, it assumes it is the coordinator and broadcasts a coordinator message to all lower id processes. If a process receives a reply to its election message, it waits for a coordinator message; if none arrives after another timeout, it starts a new election.

Example

For instance, process N3 starts an election with id 3. The message is forwarded to N32, which replaces 3 with 32 because 32 > 3. It then forwards to N5, which keeps 32 because 32 > 5, and finally to N80, which replaces 32 with 80. The message circulates back to N80, which changes it to an elected message and broadcasts it so that all processes know that N80 is the leader.

Algorithm / Steps

1. If a process detects that the coordinator has failed via a failure detector, it checks whether its own id is the highest among known processes. If so, it elects itself as coordinator and sends a coordinator message to all processes with lower ids.
2. If its id is not the highest, it initiates an election by sending an election message containing its id to all processes with higher ids.
3. Upon receiving an election message, a process compares the id in the message with its own. If the received id is larger, it forwards the message unchanged; if it is smaller and the process has not forwarded an election message before, it replaces the id with its own and forwards it.
4. If a process receives no reply to its election message within a timeout, it assumes it is the coordinator and broadcasts a coordinator message to all lower id processes.
5. If a process receives a reply to its election message, it waits for a coordinator message. If none arrives after another timeout, it starts a new election.
6. When the election message returns to the process that originally sent it, that process changes the message type to elected and circulates it so that all processes learn the leader.
7. In the case of a faulty highest-id process, higher-id processes respond with an ok message, and the highest alive process eventually times out and declares itself coordinator, then sends a coordinator message to all nodes.

Key points

- The election message carries a process’s id and is forwarded clockwise in a logical ring.
- A process with the highest id among those that have not failed becomes the coordinator and broadcasts a coordinator message to all lower-id processes.
- If a process receives no reply to its election message within a timeout, it assumes leadership and initiates a new election.
- The algorithm terminates with the highest non-faulty id process elected, ensuring safety and liveness in the absence of failures.

Primitives for Distributed Communication

Partly covered in the source.

Definition

Message passing is a communication primitive used in distributed computing, where multiple autonomous computers with their own memory communicate through messages.

Explanation

Distributed computing comprises multiple autonomous computers having their own memory and they communicate through the messages called message passing.

Key points

- Distributed computing involves multiple autonomous computers.
- Communication is achieved via message passing.

Synchronous versus Asynchronous Executions

Partly covered in the source.

Definition

Synchronous execution refers to a ring where protocols guarantee liveness, whereas asynchronous execution refers to a ring where protocols may not terminate.

Explanation

In synchronous algorithms the message complexity can be reduced to O(n) when general arithmetic operations are permitted, but if the time complexity is unbounded the complexity reverts to nlog n messages. In asynchronous rings the best-known algorithm also requires nlog n messages, and in the worst case the algorithm may send n^2 messages. The algorithm that works in both synchronous and asynchronous models may not guarantee termination in the asynchronous setting because timeouts can cause the protocol to never finish, whereas it satisfies liveness in the synchronous model.

Example

In a ring of processors with identifiers 0–3, processor p3 sends its ID to the left (p4). When p4 receives 0 it does not forward it further. p4 then sends its own ID, and when it reaches p0 its ID is greater than 3, so it is forwarded further. The message continues until it reaches p1 and then p3, which sees its own ID and declares itself the leader. This illustrates how the algorithm operates in both synchronous and asynchronous settings.

Algorithm / Steps

1. Each processor sends its own identifier to its right neighbor.
2. Upon receiving an identifier, a processor forwards it only if the received identifier is larger than its own.
3. If a processor receives its own identifier, it declares itself the leader and stops forwarding.
4. The algorithm terminates when the highest identifier has circulated the ring.

Key points

- Synchronous execution can achieve O(n) message complexity under certain conditions; otherwise it is nlog n.
- Asynchronous execution’s best-known algorithm also requires nlog n messages, and the worst case can reach n^2 messages.
- The leader election algorithm works in both synchronous and asynchronous models but may not terminate in the asynchronous case due to timeouts.

- Liveness is guaranteed in synchronous executions but not in asynchronous executions.

A Model of Distributed Computations: A Distributed Program

Explanation

- A distributed program consists of a set of asynchronous processes that communicate via message passing or shared memory.

A Model of Distributed Executions

Partly covered in the source.

Key points

- Paxos is a consensus protocol used for leader election in industry systems such as Google Chubby and Apache Zookeeper.
- Paxos guarantees safety but does not guarantee liveness; it ensures eventual liveness.
- In Google Chubby, a group of replicas (servers A, B, C, D, E) elect a leader at all times.

Models of Communication Networks

Partly covered in the source.

Definition

A communication network model in this context refers to a hierarchical arrangement of racks, switches, and storage nodes that form a datacenter architecture, often described as a 2-level or 3-tier topology with top-of-rack switches connected to a core switch.

Explanation

The model connects each rack through a top-of-rack switch, and these switches are linked to a core switch that aggregates traffic across the datacenter. Within each rack, storage backend nodes with SSDs provide data storage, while a frontend handles job submission and client requests, forming a 3-tier architecture. The architecture can be extended across multiple geographic sites, creating a geographically distributed cloud where each site contains its own datacenter structure and the sites communicate over a high-speed network. Distributed computing is achieved by autonomous computers that communicate via message passing. Bandwidth growth, as described by Gilder’s law, has historically doubled each year, leading to higher link speeds and larger disk capacities, which supports the scalability of such network models. Power consumption and cooling are critical considerations; datacenters use metrics such as annual water usage per IT equipment energy and total facility power per IT equipment power to evaluate efficiency. Cooling methods include air-suction and water-based systems that circulate purified water to absorb heat.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Example

A geographically distributed cloud that deploys the described 2-level topology across multiple data centers, each with its own rack, storage nodes, and core switch, and connects them via a fast network.

Algorithm / Steps

None provided in the passages.

Key points

- Hierarchical 2-level/3-tier topology with top-of-rack and core switches.
- Storage backend nodes with SSDs reside within racks.
- Frontend handles job submission and client requests.
- Distributed across multiple geographic sites connected by high-speed network.
- Bandwidth and storage capacities have doubled over time, enabling scalability.
- Power and cooling efficiency are measured by water usage and power ratios; cooling uses air and water methods.

Global State of a Distributed System.

Partly covered in the source.

Explanation

- The global state of a distributed system represents a snapshot of all processes and communication channels.
- Collecting local states without a global clock requires specialized snapshot protocols.

Source and licence

This PDF is adapted from **NOC:Cloud Computing and Distributed Systems (NPTEL course book, weeks 1, 3)** (https://nptel.ac.in/courses/106104182), accessed 2026-10-09.

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).